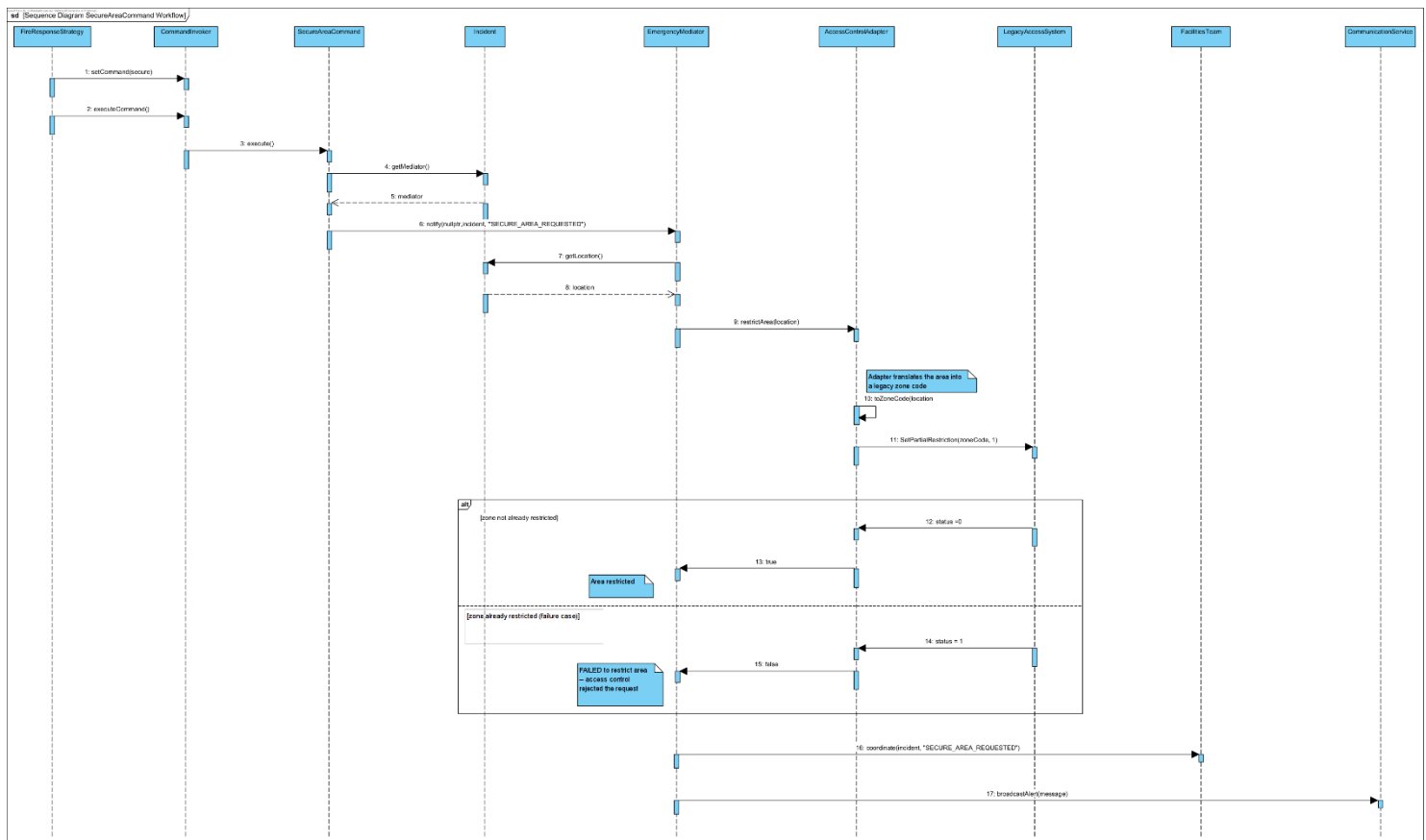


Task 4

1. Sequence diagram: SecureAreaCommand



The diagram shows what happens when a fire response needs to secure an area, done by demonstrating how Command, Mediator and Adapter work together as one flow instead of separately.

It starts when the `FireResponseStrategy`, in this case but can also be the `MedicalResponseStrategy` depending on what is initialized, builds a `SecureAreaCommand` and runs it through a `CommandInvoker`. The invoker doesn't know what the command does and just calls `execute()`, which is the command pattern inaction; the request is packaged as an object instead of being handled by a big if/else chain.

When the command executes, it doesn't try to restrict the area itself. Instead, it asks the `Incident` for its `EmergencyMediator` and notifies it that an area needs securing. This is the Mediator pattern, `SecureAreaCommand` has no idea which teams or systems exist, it just reports the event, and the mediator decides who needs to react.

The mediator then calls `restrictArea()` on the access control system. But the real access control hardware (`LegacyAccessSystem`) uses old method names and different data types (zone codes and status numbers instead of area names and true/false). That's

where the AccessControlAdapter comes in as it translates the mediator's request into something the legacy system understands and translates the legacy system's answer back into a simple true/false the mediator can use. This is the Adapter pattern; it lets two incompatible interfaces talk to each other without either side changing.

There is also a realistic failure case in the diagram where if the zone is already restricted, the legacy system rejects the request, the adapter reports the failure, and the mediator logs it as a failed restriction instead of crashing. So either way the mediator still tells the Facilities team to stand by and broadcast a message through the Communication service. Showing that the mediator keeps coordinating the response regardless of the outcome.

```

*
==1==
==1== HEAP SUMMARY:
==1==    in use at exit: 0 bytes in 0 blocks
==1==   total heap usage: 78 allocs, 78 frees, 75,930 bytes allocated
==1==
==1== All heap blocks were freed -- no leaks are possible
==1==
==1== For lists of detected and suppressed errors, rerun with: -s
==1== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
PS C:\Users\Manasse Kabongo\Desktop\courses\2nd Year\COS 214\Prac05\NewTeam-COS214>

```

Figure 1: Valgrind memory analysis showing no memory leaks or errors.

Valgrind was used with `--leak-check=full --show-leak-kinds=all`. The analysis reported 78 allocations and 78 frees, with 0 bytes still in use at program exit. Valgrind also reported 0 errors from 0 contexts, indicating that no memory leaks or invalid memory operations were detected during the execution.

Command: `docker compose run --rm campusguard valgrind --leak-check=full --show-leak-kinds=all ./campusguard`

GDB Investigation – Mediator Routing Bug

Bug investigated: During development, the Mediator initially had a routing problem where the response unit being handled needed to be determined from the dispatched unit rather than inferred from the incident type. This could cause the wrong response component to receive an incident.

The implementation was changed so that `DispatchUnitCommand` sends an event containing the actual dispatched unit, using the format `UNIT_DISPATCHED:<UnitType>`. The Mediator then extracts the unit type and routes the incident to the appropriate response component.

```

Breakpoint 1, EmergencyMediator::notify (this=0x7ffe5b896c60, sender=0x0, incident=..., event=0x564350244380
"UNIT_DISPATCHED:Security") at EmergencyMediator.cpp:22
22  {
(gdb) print event
$1 = 0x564350244380 "UNIT_DISPATCHED:Security"
(gdb) print incident.getId()
$2 = "INC-001"

```

Figure 2: GDB inspecting the dispatched unit event and incident ID.

GDB was used to inspect the values entering EmergencyMediator::notify(). The debugger showed that incident INC-001 generated the event UNIT_DISPATCHED:Security, confirming that the actual dispatched unit was available to the Mediator.

```
Breakpoint 1, EmergencyMediator::notify (this=0x7ffe5b896c60, sender=0x0, incident=..., event=0x564350244380
"UNIT_DISPATCHED:Facilities") at EmergencyMediator.cpp:22
22  {
(gdb) next
23      std::string evt(event);
(gdb) next
25      const std::string dispatchPrefix = "UNIT_DISPATCHED:";
(gdb) next
27      if (evt.compare(0, dispatchPrefix.size(), dispatchPrefix) == 0)
(gdb) next
29          std::string unitType = evt.substr(dispatchPrefix.size());
(gdb) next
31          if (unitType == "Medical")
(gdb) next
35              else if (unitType == "Security")
(gdb) next
39              else if (unitType == "Facilities")
(gdb) next
41                  facilities->handleIncident(incident);
(gdb) next
[FACILITIES] Campus Facilities managing building systems for incident INC-002 at Engineering Building.

Breakpoint 1, EmergencyMediator::notify (this=0x7ffe5b896c60, sender=0x7ffe5b896dd0, incident=..., event=0x56
4342732798 "FACILITIES_READY") at EmergencyMediator.cpp:22
22  {
(gdb) next
```

Figure 3: GDB stepping through Mediator routing of the Facilities dispatch.

GDB Investigation: During development, a routing issue was identified in the Mediator where response-unit handling needed to be based on the actual dispatched unit. GDB was used to inspect the event received by EmergencyMediator::notify(). The debugger showed events such as UNIT_DISPATCHED:Security and UNIT_DISPATCHED:Facilities. Stepping through the Mediator confirmed that the unit type was extracted from the event and used to select the appropriate response component. For the Facilities case, execution reached the Facilities branch and facilities->handleIncident(incident) was called for incident INC-002.

Result: The investigation confirmed that the corrected Mediator routing uses the dispatched unit contained in the event rather than relying on the incident type. The behaviour was verified for both Security and Facilities dispatches during the two end-to-end response scenarios.

Command: docker compose run --rm campusguard bash -c "g++ -std=c++11 -g *.cpp -o campusguard && gdb ./campusguard"